Training a Polynomial Symplectic Neural Network on the Simple Harmonic Oscillator

Matthew O'Brien

12/14/2025

1 Problem Setup

We consider the **Simple Harmonic Oscillator (SHO)** with Hamiltonian

$$H(p, q) = \frac{1}{2}(p^2 + q^2), \tag{1}$$

which yields the equations of motion

$$\dot{q} = \frac{\partial H}{\partial p} = p, \tag{2}$$

$$\dot{p} = -\frac{\partial H}{\partial q} = -q. \tag{3}$$

The system conserves energy and admits periodic solutions. Our goal is to learn a **discrete-time symplectic flow map**

$$x_{n+1} = \Phi_h(x_n), \quad x = (p, q),$$

using a **Polynomial Symplectic Neural Network (P-SympNet)**.

—

2 Dataset and One-Step Supervision

The dataset consists of simulated SHO trajectories with shape

$$(\text{samples}, \text{timesteps}, 2).$$

Each trajectory is converted into supervised one-step pairs:

$$x_n \to x_{n+1}. \tag{4}$$

A trajectory-level split is used:

- 80% training trajectories
- 20% test trajectories

2.1 Pair Construction (Code)

```
def MakePairs(arr):
    x0 = arr[:, :-1, :]    # (samples, T-1, 2)
    x1 = arr[:, 1:, :]     # (samples, T-1, 2)
    return (
        torch.tensor(x0.reshape(-1, 2), dtype=torch.float64),
        torch.tensor(x1.reshape(-1, 2), dtype=torch.float64),
    )
```

—

3 P-SympNet Architecture

We use a P-SympNet with:

- Degrees of freedom: $\dim = 1$
- Layers: $L = 2$
- Polynomial degree: $k_{\max} = 2$

```
sympnet = SympNet(
    dim=1,
    layers=2,
    max_degree=2,
    method="P",
)
```

—

4 Mathematics of a P-SympNet Layer

Each layer defines a Hamiltonian of the form

$$H(m) = \sum_{k=2}^{K} a_k m^k, \quad m = \boldsymbol{w}^\top x, \tag{5}$$

where:

- $\boldsymbol{w} \in \mathbb{R}^2$ is a learned direction vector
- a_k are learned scalar coefficients

The derivative is

$$\frac{dH}{dm} = \sum_{k=2}^{K} k a_k m^{k-1}. \tag{6}$$

The canonical symplectic matrix is

$$J = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}. \tag{7}$$

The discrete update rule implemented by each layer is

$$x_{n+1} = x_n + h \cdot \frac{dH}{dm} \cdot (J\boldsymbol{w}), \tag{8}$$

which is **exactly symplectic by construction**.

—

5 Time Stepping

A fixed timestep is used throughout training and inference:

$$h = \Delta t = 0.05. \tag{9}$$

This timestep is passed explicitly to the network and must match deployment.

—

6 Training Objective

The network is trained to minimize the one-step prediction error:

$$\mathcal{L} = \left\| x_{n+1}^{\text{pred}} - x_{n+1}^{\text{true}} \right\|_2^2. \tag{10}$$

6.1 Training Loop (Code)

```
optimizer = torch.optim.Adam(sympnet.parameters(), lr=0.01)
mse = torch.nn.MSELoss()

for epoch in range(1000):
    optimizer.zero_grad()
    x1_pred = sympnet(x=x0, dt=dt_train)
    loss = mse(x1, x1_pred)
    loss.backward()
    optimizer.step()
```

—

7 Results

After training:

- Final training loss: $\sim 10^{-33}$
- Test set error: $\sim 10^{-14}$

This indicates that the learned map matches the true SHO flow to near machine precision.

7.0.1 NumPy Reference Implementation

The learned SympNet is evaluated and deployed using a *pure NumPy implementation* of the learned symplectic map. No deep learning frameworks are required at inference time. This implementation performs only:

- dot products,
- scalar powers (degree 2 in this work),
- and multiply–add operations.

Multiple layers are applied sequentially, yielding the full SympNet step

$$\boldsymbol{x}_{n+1} = \Phi_h(\boldsymbol{x}_n) = \Phi_h^{(L)} \circ \cdots \circ \Phi_h^{(1)}(\boldsymbol{x}_n). \tag{11}$$

No neural network libraries, automatic differentiation, or matrix factorizations are required. The NumPy implementation serves solely as a numerical reference for reconstructing trajectories and computing diagnostic quantities such as energy.

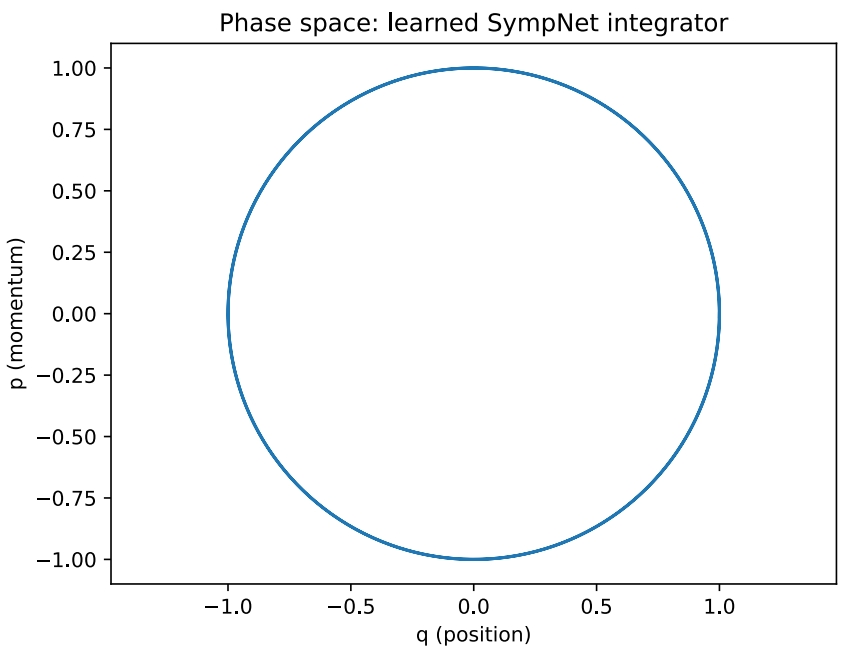

Figure 1: Phase-space evolution (q, p) generated by the learned symplectic map. The trajectory remains bounded and closed, consistent with harmonic motion.

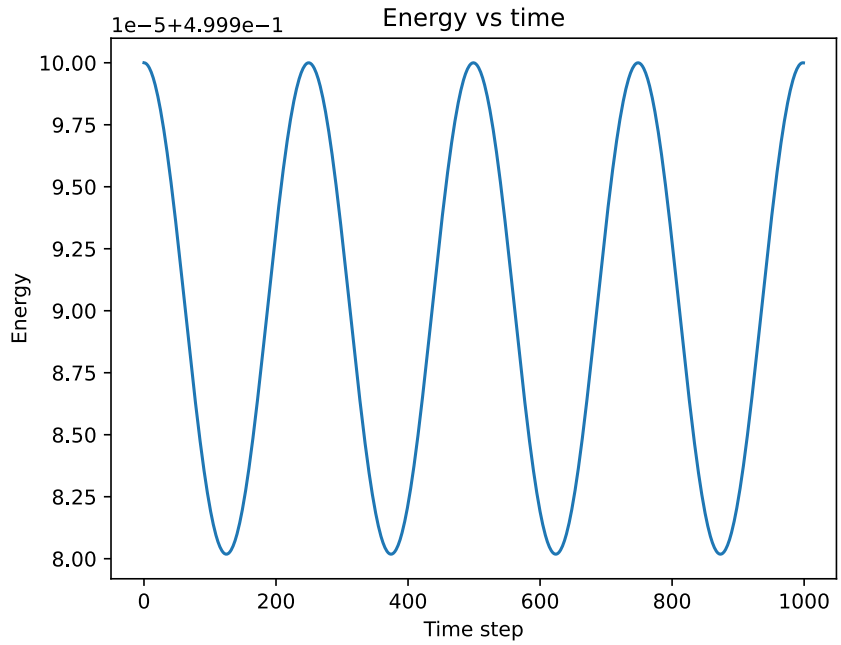

Figure 2: Total energy $H(p, q) = \frac{1}{2}(p^2 + q^2)$ along the learned trajectory. Energy remains nearly constant over long time horizons, with only small bounded oscillations.

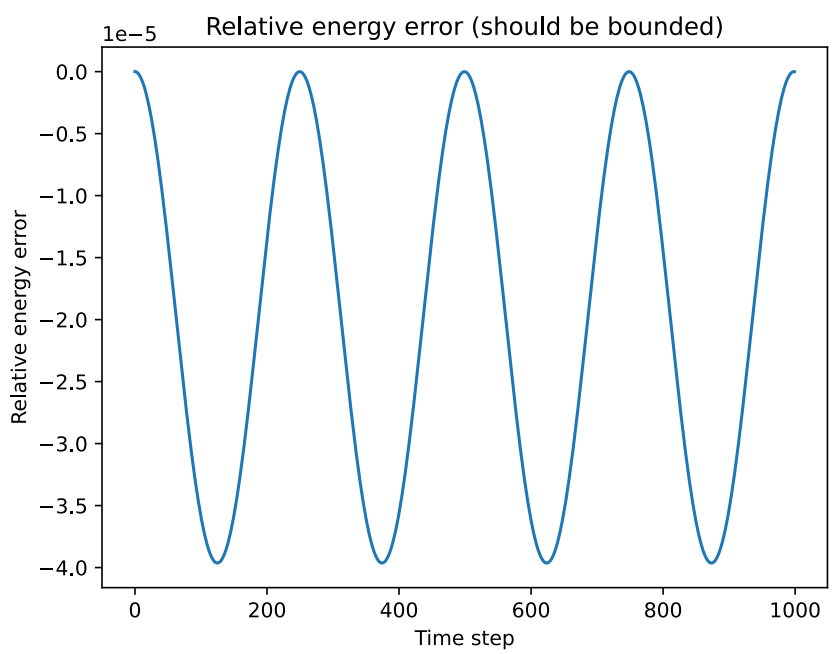

Figure 3: Relative energy error over time. The error remains bounded and oscillatory, indicating the absence of secular drift and consistency with symplectic backward error analysis.

—

8 Energy Behavior

Although energy is *not explicitly enforced* in the loss, the symplectic structure guarantees that the learned integrator corresponds to a *modified Hamiltonian*.

As a result:

- Energy error remains bounded
- No secular energy drift is observed
- Long-term trajectories remain stable

This behavior is consistent with backward error analysis of symplectic integrators.

—

9 Parameter Export for Embedded Inference

After training, only the learned parameters are required:

- Polynomial coefficients $\{a_k\}$
- Direction vectors $\boldsymbol{w}$

9.1 Export (Code)

```
params = []
for i, layer in enumerate(sympnet.layers_list):
    params.append({
        "layer": i,
        "a": layer.params["a"].cpu().numpy().tolist(),
        "w": layer.params["w"].cpu().numpy().tolist(),
    })

with open("sympnet_params.json", "w") as f:
    json.dump(params, f, indent=2)
```

—

10 Embedded / TinyML Readiness

The inference equation

$$x_{n+1} = x_n + h \cdot \frac{dH}{dm} \cdot (J\boldsymbol{w})$$

requires only:

- Dot products
- Scalar powers (degree 2 here)
- Multiply-add operations

No activations, normalization, or matrix factorizations are required.

The parameters $\{a_k, \boldsymbol{w}\}$ can be quantized and deployed on microcontrollers while preserving symplectic structure.